PL-SQL Lab 1

Name : Devananda T C

Roll No : AM.SC.U4AIE23111

1. Do all the questions discussed in the class
   a) Write a function which accepts 2 integer values and return their sum.

plsql/postgres@PostgreSQL 14

Query Editor | Query History

```
1  create or replace function sum(a INT, b INT)
2  RETURNS INT AS $$
3  BEGIN
4      RETURN a + b;
5  END;
6  $$ LANGUAGE plpgsql;
7
8  select sum(1123,256);
```

Data Output | Messages | Notifications | Explain

	sum integer
1	1379

   b) Write a function which accepts an integer value and returns if it is odd or even

Query Editor | Query History

```
1  create or replace function odd_even(a integer)
2  returns varchar
3  as $$
4  begin
5      if mod(a, 2) = 0 then
6          return 'even';
7      else
8          return 'odd';
9      end if;
10 end;
11 $$ language plpgsql;
12
13 select odd_even(11)
14
```

Data Output | Messages | Notifications | Explain

	odd_even character varying
1	odd

c) Write a function which accepts employee number and returns salary of that employee

```
create or replace function emp_sal(empno Emp.eno%type)
returns Emp.sal%type as
$$
   Declare
      salary emp.sal%type;
   Begin
    select sal into salary from emp
    where eno = empno;
    return salary;
   End;
$$
Language 'plpgsql';

select emp_sal(103) as salary;
```

Data Output | Messages | Notifications

	salary integer
1	60000

d) Write a function which accepts a dept no. and finds the number pf employees in that department

```
create or replace function emp_count(dept_no Emp.dno%type)
        returns integer as
$$
        Declare
                cnt integer;
        Begin
                select count(*) into cnt from Emp
                where dno = dept_no;

                return cnt;
        End;
$$
Language 'plpgsql';

select emp_count(10) as total_employees;
```

Data Output | Messages | Notifications

	total_employees integer
1	1

e) Write a function which takes an employee number and give that employee 10% hike in salary , if average salary of the corresponding department is greater than employees current salary.

Query Query History

```
create or replace function emp_hike(emp_no emp.eno%type)
       returns emp.sal%type as
$$
   declare
      salary emp.sal%type;
    begin
      select sal into salary from emp where eno = emp_no;
          if salary < (select avg(sal) from emp
           where dno = (select dno from emp where eno = emp_no)) then
                update emp
                set sal = sal + (sal * 0.10)
                where eno = emp_no;
                select sal into salary from emp where eno = emp_no;
           end if;
           return salary;
    end;
$$
language 'plpgsql';

select emp_hike(2) as updated_salary;
```

Data Output Messages Notifications

	updated_salary integer
1	[null]

f) Write a function which accepts an employee name and returns the department name of that employee.

```
create or replace function get_deptname(emp_name emp.ename%type)
       returns dept.dname%type as
$$
   declare
       deptname dept.dname%type;
    begin
        select dname into deptname from dept
         where dno = (select dno from emp where ename = emp_name);
              return deptname;
     end;
$$
language plpgsql;

select get_deptname('Arun') as department_name;
```

Data Output Messages Notifications

	department_name character varying
1	HR

2. Consider the following schema .

Emp(eno, ename, sal)

EMP_PROJ(eno, pno, prjt_hrs).

a) Write a function ProjectLoad that returns the total project working hours for the given eno.

```
create or replace function projectload(emp_no emp.eno%type)
        returns integer as
$$
        declare
                total_hrs integer;
        begin
                select sum(prjt_hrs) into total_hrs
                from emp_proj
                where eno = emp_no;

                return total_hrs;
        end;
$$
language plpgsql;

select projectload(101) as total_hours;
```

Output Messages Notifications

total_hours integer
8

b) Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary. New salary is current sal + difference between currrent sal and average salary.

```
create or replace function update_sal(emp_no emp.eno%type)
        returns varchar as
$$
   Declare
       avg_sal    numeric;
       curr_sal   numeric;
       new_sal    numeric;

    Begin
       select sal into curr_sal from emp where eno = emp_no;
       select avg(sal) into avg_sal from emp;

       if curr_sal < avg_sal then
               update emp
                set sal = sal + (avg_sal - curr_sal)
                where eno = emp_no;
        end if;
           select sal into new_sal from emp where eno = emp_no;
           return new_sal;

    end;
$$
Language 'plpgsql';

select update_sal(3);
```

Data Output Messages Notifications

	update_sal character varying
1	[null]

2. Consider the following tables

Item(ino, iname, unit_price)
Transaction(tr_no,ino,qty)
Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```
create table item(ino  int primary key,
iname  varchar(50),
unit_price  numeric);
create table transactions(tr_no   int,
ino  int,qty  int);
insert into item values
(101, 'Pen', 10),
(102, 'Book', 50),
(103, 'Bag', 300),
(104, 'Pencil', 5);
insert into transactions values
(1, 101, 2),
(2, 101, 3),
(3, 102, 1),
(4, 103, 4),
(5, 103, 2);
```

```
create or replace function delete_item_if_less(emp_ino item.ino%type)
returns varchar as
$$
declare
    trans_count numeric;
begin
    select count(*) into trans_count
    from transactions
    where ino = emp_ino;

    if trans_count < 2 then
        delete from item
        where ino = emp_ino;
    end if;

    return 'Done';
end;
$$
language 'plpgsql';

select delete_item_if_less(101);
```

Data Output | Messages | Notifications

	delete_item_if_less character varying
1	Done

3. Consider a Bank database which includes the following tables.
   ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)
BRANCHES(br_no, br_name, loc)
CUSTOMER(cno, cname, c_type)

Query | Query History

```sql
create table customer(cno  int primary key,
cname   varchar(50),
c_type  varchar(5));
create table branches(br_no  int primary key,
br_name  varchar(50),
loc      varchar(50));
create table accounts(ac_no int primary key,
br_no int, cust_no  int, ac_type  varchar(20),
bal  numeric,
foreign key (cust_no) references customer(cno),
foreign key (br_no) references branches(br_no));
insert into customer values
(101, 'Arun', ' '),
(102, 'Meera', ' '),
(103, 'Vivek', ' ');
insert into branches values
(1, 'Main Branch', 'Kochi'),
(2, 'City Branch', 'Chennai'),
(3, 'Mall Road', 'Bangalore');
insert into accounts values
(1001, 1, 101, 'Savings', 40000),
(1002, 1, 101, 'Current', 20000),
(1003, 2, 102, 'Savings', 30000),
(1004, 2, 102, 'Current', 15000),
(1005, 3, 103, 'Savings', 80000);
```

a) Write a function that accepts a threshold value and a customer number . The program updates the c_type based on the threshold value. Ie. If balance> threshold then class A, else class B.

```sql
create or replace function update_ctype(threshold numeric, custno customer.cno%type)
returns varchar as
$$
declare
    total_bal numeric;
begin
    select sum(bal) into total_bal
    from accounts
    where cust_no = custno;

    if total_bal > threshold then
        update customer set c_type='A' where cno=custno;
    else
        update customer set c_type='B' where cno=custno;
    end if;

    return 'Done';
end;
$$
language plpgsql;

select update_ctype(50000, 101);
```

Data Output | Messages | Notifications

SQL

	update_ctype character varying
1	Done

b) Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.

Query Query History

```
--4(b)--
create or replace function CloseBranch(close_br branches.br_no%type,
                                       new_br branches.br_no%type)
returns varchar as
$$
declare
    acc_count int;
begin
    update accounts
    set br_no = new_br
    where br_no = close_br;

    delete from branches
    where br_no = close_br;
    return 'Done';
end;
$$
language plpgsql;

select CloseBranch(1, 2);
```

Data Output Messages Notifications

	closebranch character varying
1	Done

c) Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

Query Query History

```
--Safe Withdraw--
create or replace function safe_withdraw(accno accounts.ac_no%type,
amt numeric)
returns varchar as
$$
declare
    curr_bal numeric;
begin
    select bal into curr_bal
    from accounts
    where ac_no = accno;

    if curr_bal >= amt then
        update accounts
        set bal = curr_bal - amt
        where ac_no = accno;

        return 'Success';
    else
        return 'Insufficient Funds';
    end if;

end;
$$
language plpgsql;

select safe_withdraw(1001, 20000);
```

Data Output Messages Notifications

	safe_withdraw character varying
1	Success